CNN Basics

Structured Notes

Convergence Behavior, Gradient

Monitoring, Underfitting & Overfitting,
Bias-Variance Tradeoff, Model Capacity, and ANN Use Cases

Contents

1 Convergence Behavior in Neural Network Training **2**
1.1 Failure Modes . . . 2
1.2 Causes of Training Failure . . . 2

2 Inspecting and Monitoring Gradient Norms **2**
2.1 Why Monitor Gradient Norms . . . 3
2.2 Tracking Gradient Norms in PyTorch . . . 3
2.3 Interpreting Gradient Norm Curves . . . 4

3 Underfitting and Overfitting **4**
3.1 Training Error vs. Validation Error . . . 4
3.2 Underfitting . . . 4
3.3 Overfitting . . . 5
3.4 Effect of Epochs on Training . . . 5

4 Bias-Variance Tradeoff **5**

5 Model Capacity **5**
5.1 Depth vs. Width . . . 6
5.2 Capacity and Bias-Variance . . . 6

6 Practical Training: Validation Loss Analysis **6**
6.1 Data Preparation . . . 6
6.2 Model Architecture Using nn.Sequential . . . 7
6.3 Training Loop with Validation Tracking . . . 7
6.4 Hyperparameter Tuning Observations . . . 8

7 ANN Use Cases **8**
7.1 Regression Problems . . . 8
7.2 Classification Problems . . . 8
7.3 Common Ground Between Regression and Classification . . . 9
7.4 Fraud Detection . . . 9
7.5 Healthcare Prediction . . . 9
7.6 Image and Speech Applications . . . 9

8 The Complete ANN Pipeline **9**
8.1 ANN as Nested Functions . . . 10
8.2 How Layers Share Responsibility . . . 10
8.3 Expressivity vs. Generalization . . . 10

9 The Core Equation of Deep Learning **10**
9.1 Conceptual Check Questions . . . 11

10 Summary **11**

1 Convergence Behavior in Neural Network Training

Convergence Behavior

Convergence behavior describes how a neural network's loss function evolves during training. In the **ideal case**, the training loss decreases smoothly, the gradient gradually shrinks, and the model approaches a minimum.

1.1 Failure Modes

Training does not always proceed ideally. There are three primary failure modes:

1. **Divergence** — The loss explodes. The model's parameters move farther from any optimum with each update.
2. **Oscillation** — The loss fluctuates up and down without settling. The model cannot stabilize on a solution.
3. **Plateau** — The loss stops decreasing. The model gets stuck at a point where gradients are near zero but the loss is still high.

1.2 Causes of Training Failure

Cause	Description
Learning rate too large	The model skips over the minimum value during parameter updates, causing divergence or oscillation.
Poor initialization	Weights initialized to bad values can cause the network to start in a region of the loss surface that is difficult to escape.
Vanishing gradients	Gradients become extremely small during backpropagation, causing the network to stop learning.
Ill-conditioned loss surface	The wrong loss function is used to calculate the loss. If the loss surface is ill-conditioned, the loss function must be changed entirely.

Key Distinction

The first three causes (learning rate, initialization, vanishing gradients) are still manageable through hyperparameter tuning. However, an **ill-conditioned loss surface** requires changing the loss function itself — this is a fundamental problem that cannot be fixed by tuning other parameters.

A large learning rate causes divergence because it skips the minimum value. An extremely small learning rate leads to very slow convergence. These are standard diagnostic questions:

- Why can a large learning rate cause divergence? It skips the minimum value.
- What happens if the learning rate is extremely small? Very slow convergence.
- If gradient norm becomes very small early in training, what might be happening? Vanishing gradient problem.

2 Inspecting and Monitoring Gradient Norms

Gradient Norm

The **gradient norm** is the magnitude (not the vector direction) of the gradient of the loss with respect to the model parameters. Monitoring gradient norms during training helps diagnose optimization issues such as vanishing or exploding gradients.

2.1 Why Monitor Gradient Norms

During training, gradients can be inspected using the computational graph. Tracking the gradient norm helps to:

- Determine whether the model is converging properly
- Detect vanishing gradients (gradient norm becomes very small early in training)
- Detect exploding gradients (gradient norm spikes or shows zigzag patterns)
- Debug whether the loss function or optimizer is correct

2.2 Tracking Gradient Norms in PyTorch

Gradient Norm Tracking

```python
import torch
import torch.nn as nn

# Create a simple model
model = nn.Sequential(
    nn.Linear(1, 8),
    nn.ReLU(),
    nn.Linear(8, 1)
)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# Synthetic data: y = x^2
x = torch.linspace(-1, 1, 100).reshape(-1, 1)
y = x ** 2

gradient_norms = []
for epoch in range(100):
    pred = model(x)
    loss = nn.MSELoss()(pred, y)
    optimizer.zero_grad()
    loss.backward()

    # Track gradient norm
    for param in model.parameters():
        if param.grad is not None:
            gradient_norms.append(
                param.grad.norm().item()
            )
    optimizer.step()
```

Listing 1: Tracking gradient norms during training

The gradient norm is used because it captures the **magnitude** of the gradient, not the individual vector components. This gives a single scalar value that summarizes how strongly the loss is responding to parameter changes.

2.3 Interpreting Gradient Norm Curves

Pattern	Interpretation
Decreasing then stabilizing	Normal behavior. Gradients are high initially, decrease rapidly, then stabilize near zero as the model converges.
Spike or zigzag pattern	Exploding gradients. The model is unstable and parameters are being updated too aggressively.
Near-zero from the start	Vanishing gradient problem. The model cannot learn because gradients are too small from the beginning.

Vanishing vs. Normal Gradient Decay

All gradients will eventually vanish or stabilize after sufficient epochs because repeated differentiation naturally reduces gradient magnitudes. This is **normal**. The **problem** occurs when gradients vanish **initially**, in the first few epochs — this indicates a vanishing gradient problem (e.g., caused by sigmoid activation).

Using `sigmoid` activation, gradients vanish very quickly based on the value of the input. This is a well-known limitation of sigmoid in deep networks.

3 Underfitting and Overfitting

Generalization

Training minimizes the loss on the training data, but the true goal is to perform well on **unseen data**. The ability of a model to perform well on unseen data is called **generalization**.

3.1 Training Error vs. Validation Error

Training Error	Validation Error	Diagnosis
Low	Low	Good model (well generalized)
Low	High	Overfitting
High	High	Underfitting

Validation error is the more important metric. Training error can be reduced by simply increasing model complexity, but validation error measures actual generalization ability on data the model has not seen.

3.2 Underfitting

Underfitting

Underfitting occurs when the model is too simple and cannot capture the underlying pattern in the data. It is characterized by **high training loss** and **high validation loss**. Underfitting corresponds to **high bias**.

For example, using a linear model to fit $y = x^2$ results in underfitting — the linear model cannot capture the curvature. With very few neurons, the model becomes too simple and underfits.

Solutions to underfitting:

- Increase the complexity of the model (add more neurons)
- Increase the depth of the network (add more layers)
- Increase model width (neurons per layer)

3.3 Overfitting

Overfitting

Overfitting occurs when the model is too complex and fits the noise in the training data rather than the underlying pattern. It is characterized by **very low training loss** but **high validation loss**. The model **memorizes** instead of **learning**. Overfitting corresponds to **high variance**.

If the model is made too complex by increasing depth and width, it starts to memorize the data instead of learning the pattern. Training error can be reduced by increasing neurons, but this does not guarantee better validation performance.

Underfitting is Easier to Fix

Underfitting is much easier to tackle than overfitting. Increasing model capacity reliably addresses underfitting, but reducing overfitting requires more nuanced approaches such as regularization, dropout, early stopping, or obtaining more data.

3.4 Effect of Epochs on Training

Increasing the number of epochs causes the model to see the training data again and again, which leads to **overfitting**. The model essentially becomes overtrained on the same data. Epochs can be controlled effectively using **early stopping**.

4 Bias-Variance Tradeoff

Bias-Variance Tradeoff

- **Bias**: Error from incorrect assumptions. A model with high bias is too simple and makes systematically wrong predictions.
- **Variance**: Error from sensitivity to data fluctuations. A model with high variance fits too much noise and performs inconsistently on different datasets.

The **goal** is to find a balance between bias and variance. This is the core of optimization.

The relationship between model complexity and error types:

- **Increasing model capacity** → reduces bias (solves underfitting) but increases the risk of overfitting (higher variance)
- **Decreasing model capacity** → reduces variance but increases bias

The optimal model minimizes the **total error** = bias^2 + variance + irreducible error.

Assumptions lead to biasness — incorrect assumptions cause incorrect predictions or classifications. The bias-variance tradeoff is what optimization is fundamentally about: finding the right balance where neither bias nor variance dominates.

5 Model Capacity

Model Capacity

The **capacity** of a model is its ability to fit complex functions. Capacity is determined by:

- **Number of layers** (depth)
- **Number of neurons per layer** (width)

Too low capacity leads to underfitting; too high capacity leads to overfitting.

5.1 Depth vs. Width

Depth Creates More Complexity Than Width

Increasing the **depth** (number of layers) makes the model more complex than increasing the **width** (neurons per layer). A network with a single layer of 64 neurons is **less complex** than a network with two layers of 16 neurons each. This is because backpropagation must process through two layers of functions — the output of the first layer becomes the input of the second layer, effectively changing dimensions and increasing computational complexity.

Aspect	Increasing Width	Increasing Depth
Complexity	Moderate increase	Higher increase
Backpropagation	Single layer computation	Multi-layer chain rule
Dimension change	Same space	Transforms between spaces
Risk	Lower overfitting risk	Higher overfitting risk

5.2 Capacity and Bias-Variance

- Increasing model capacity **reduces bias** and addresses underfitting
- Increasing model capacity **increases the risk of overfitting** due to higher variance
- Even a very small network can overfit if the dataset is simple or small — it depends on the data, not just the architecture

Key Insight: Good training loss does **not** guarantee good generalization. A model can achieve very low training loss while still performing poorly on unseen data (overfitting).

6 Practical Training: Validation Loss Analysis

6.1 Data Preparation

A synthetic dataset is created with 200 data points: $y = x^2$ with added noise. The data is split into 75% training and 25% validation using direct indexing (not `train_test_split`).

Data Generation with Noise

```
import torch

# 200 data points from -1 to 1
x = torch.linspace(-1, 1, 200).reshape(-1, 1)
y_true = x ** 2

# Add noise using rand_like
# rand_like returns tensor with same size as input,
# random numbers from normal distribution (mean=0, var=1)
y_noisy = y_true + 0.2 * torch.randn_like(y_true)
```

```
11
12 # 75-25 split using indexing
13 x_train, x_val = x[:150], x[150:]
14 y_train, y_val = y_noisy[:150], y_noisy[150:]
```

Listing 2: Creating synthetic data with noise

`torch.randn_like(input)` returns a tensor with the same size as the input filled with random numbers from a normal distribution with mean 0 and variance 1. Multiplying by 0.2 introduces approximately 20% noise.

6.2 Model Architecture Using nn.Sequential

Model Definition

```
1 import torch.nn as nn
2
3 # nn.Sequential allows direct definition without a class
4 model = nn.Sequential(
5     nn.Linear(1, 64),
6     nn.ReLU(),
7     nn.Linear(64, 1)
8 )
9
10 # nn.Module requires defining a class with forward()
11 # nn.Sequential can be used directly - just pass layers
```

Listing 3: Defining model with nn.Sequential

nn.Module vs. nn.Sequential

`nn.Module` requires defining a complete class with a `forward()` method. `nn.Sequential` allows direct definition by simply passing layers in sequence. For simple architectures, `nn.Sequential` is more convenient.

6.3 Training Loop with Validation Tracking

Training with Validation

```
1 optimizer = torch.optim.SGD(model.parameters(), lr=0.09)
2 criterion = nn.MSELoss()
3
4 train_losses = []
5 val_losses = []
6
7 for epoch in range(150):
8     # Training
9     model.train()
10     pred_train = model(x_train)
11     loss_train = criterion(pred_train, y_train)
12     optimizer.zero_grad()
13     loss_train.backward()
14     optimizer.step()
15     train_losses.append(loss_train.item())
16
17     # Validation
18     model.eval()
19     with torch.no_grad():
20         pred_val = model(x_val)
```

```
        loss_val = criterion(pred_val, y_val)
        val_losses.append(loss_val.item())
```

Listing 4: Training loop tracking both losses

6.4 Hyperparameter Tuning Observations

Experiments with different architectures and hyperparameters revealed the following patterns:

Configuration	Result	Observation
64 neurons, 2 layers	Overfitting	Validation loss higher than training loss
32 neurons, 2 layers	Better, still overfitting	Gap between losses reduced but not converged
3 layers (16, 16, 16)	Loss increased	Three layers not helpful, gap burst
2 layers, LR=0.09	Converged	Best configuration for this data

Early Stopping

Early stopping always works on the validation loss, not the training loss. Training loss gets stabilized very early because of the training data. Validation loss is the main metric — as long as validation loss keeps reducing, training should continue. When validation loss starts increasing while training loss continues to decrease, that signals overfitting and training should stop.

Finding the right balance is a difficult task, especially with real-world noisy data. The process involves iterative experimentation with architecture, learning rate, and number of epochs.

7 ANN Use Cases

Artificial neural networks are used whenever:

- The relationship between input and output is **complex**
- Data is **high-dimensional**
- Linear models are **insufficient**

7.1 Regression Problems

Regression with ANNs

Goal: Predict continuous values.

Examples: House price prediction, demand forecasting, energy consumption estimation, healthcare risk scoring.

Output layer: Identity (linear) activation function — because regression requires producing continuous values, not bounded outputs.

Loss function: Mean Squared Error (MSE).

7.2 Classification Problems

Classification with ANNs

Goal: Predict discrete categories.

Binary classification examples: Fraud vs. not fraud, disease vs. healthy.

Multi-class examples: Digit recognition, product category classification.

Output layer: Sigmoid for binary classification, softmax for multi-class.

Loss function: Cross-entropy.

MSE is Not Suitable for Classification

MSE is not suitable for classification tasks because classification requires discrete/distinct values, while MSE produces continuous values that are appropriate for prediction (regression) but not for classification.

7.3 Common Ground Between Regression and Classification

Everything is common between regression and classification networks **except the last layer**. The hidden layers, linear layers, activation functions, loss computation mechanism, and backpropagation are identical. The only difference is:

- **Regression**: Last layer produces continuous values (linear activation)
- **Classification**: Last layer produces discrete/distinct values (sigmoid or softmax)

7.4 Fraud Detection

Input features may include transaction amount, time of transaction, location, and user behavior history. ANNs learn non-linear relationships between user behavior patterns and fraud probability. Fraud patterns are **rarely linearly separable**, making ANNs a good choice. Traditional alternatives include decision trees and logistic regression.

7.5 Healthcare Prediction

Examples include disease risk prediction, survival probability, and hospital readmission risk. Inputs may include lab measurements, demographic features, and medical history. ANNs model complex interactions between biological variables that cannot be captured by simple linear models.

7.6 Image and Speech Applications

CNNs and RNNs are specialized forms of ANNs, but their foundation is still the basic ANN architecture: matrix multiplication plus non-linear activation. Examples include image recognition, speech recognition, face detection, and autonomous vehicles. The core idea remains: matrix multiplication + non-linearity + loss minimization.

Traditional ML Still Matters

Even after learning deep learning, traditional machine learning methods (SVM, logistic regression, random forest) remain valuable. Hybrid models that combine deep learning with traditional approaches are possible and effective. Deep learning emerged from traditional machine learning — the fundamental concept is still linear regression with matrix transformation, enhanced by activation functions.

8 The Complete ANN Pipeline

Complete ANN Pipeline

Every deep learning model — CNN, RNN, Transformer, LSTM — follows exactly this pipeline:

1. **Matrix Transformation**: Input data is transformed through weight matrices
2. **Non-linearity**: Activation functions introduce non-linear behavior

3. **Loss Computation**: Compare predictions with actual targets
4. **Backpropagation**: Compute gradients using chain rule
5. **Parameter Update**: Adjust weights using gradient and learning rate
6. **Generalization Check**: Evaluate on validation data
7. **Repeat** until convergence

The complexity varies between architectures in the matrix transformation step and loss computation, but the fundamental pipeline is identical.

8.1 ANN as Nested Functions

A neural network is a **composition of functions**:

$$\hat{y} = f_n(f_{n-1}(\dots f_2(f_1(\mathbf{x}))))$$

The loss function operates on this composition:

$$\mathcal{L} = L(f_n(f_{n-1}(\dots f_1(\mathbf{x}))), \mathbf{y})$$

Backpropagation applies the **chain rule** to compute gradients through this nested structure.

Deep Learning Defined

Deep learning is a structured chain rule over matrix operations. There is nothing "black box" inside deep learning mathematically. The only "black box" aspect is that interpretation of what each neuron has learned is difficult, but the mathematics is fully transparent.

8.2 How Layers Share Responsibility

- The **output layer** directly compares prediction with target
- **Hidden layers** are indirectly responsible — they adjust internal feature representations
- Each layer answers the question: "How should I change so that the final prediction improves?"
- This information is encoded in the **gradient magnitude**

8.3 Expressivity vs. Generalization

Increasing capacity (more neurons, more layers) increases function approximation power but also increases the risk of overfitting and high variance. Training success requires a balance of:

- Sufficient model **capacity**
- Proper **optimization** (appropriate learning rate)
- **Sufficient data**

9 The Core Equation of Deep Learning

Everything in deep learning reduces to **linear algebra** + **calculus**:

- **Linear algebra**: Matrix multiplication, vector space transformation
- **Calculus**: Chain rule, gradient computation, optimization

Together they enable **non-linear function learning**.

Probability and statistics contribute to identifying the weights and making predictions, which is also ultimately calculus.

9.1 Conceptual Check Questions

These are fundamental questions about ANN training:

1. **Why is non-linearity essential?** Linear models fail to capture complex patterns.
2. **Why is loss a function of all parameters?** The final prediction comes from the weights of all hidden layers. Every weight from the first layer to the last contributes to the predicted value. The final loss is the contribution of all parameters.
3. **What does gradient magnitude tell us?** It reflects the **sensitivity of loss to parameter change**. The gradient indicates how strongly a particular parameter influences the loss.
4. **Why can a model with low training loss still perform poorly?** Due to overfitting (high variance) — the model memorized training data but cannot generalize.
5. **What mathematical tool enables backpropagation?** The **chain rule** from calculus.

10 Summary

- **Convergence behavior**: Ideal training shows smooth loss decrease; failure modes include divergence, oscillation, and plateau
- **Gradient monitoring**: Track gradient norms to detect vanishing/exploding gradients; gradients should decrease then stabilize, not vanish immediately
- **Underfitting**: High bias, model too simple — increase capacity to fix
- **Overfitting**: High variance, model too complex, memorizes noise — harder to fix than underfitting
- **Bias-variance tradeoff**: Finding the balance between model simplicity and complexity is the core of optimization
- **Model capacity**: Depth adds more complexity than width; a single-layer 64-neuron network is less complex than a two-layer 16-16 network
- **Early stopping**: Based on validation loss, not training loss
- **ANN use cases**: Regression (linear output, MSE), classification (sigmoid/softmax output, cross-entropy), fraud detection, healthcare
- **Complete pipeline**: Matrix transformation $\rightarrow$ non-linearity $\rightarrow$ loss $\rightarrow$ backprop $\rightarrow$ update $\rightarrow$ generalize $\rightarrow$ repeat
- **Deep learning**: Structured chain rule over matrix operations — mathematically transparent, interpretability is the only "black box"